OWL-ORCA MASTER INSTALLER v7.1.0

Zero-Downtime Edition

Three-Pass Deep Audit Report

Error Examination | Fix Suggestions | Enhancements with Constraints

Document: install.sh (2,692 lines)
Audit Scope: 3 Passes: Errors, Fixes, Enhancements
Key Topics: Stream Racing, Protocol Translation, Radix Tree, Circuit Breakers, SIGHUP
Specific Questions: Claude Extended Thinking, OpenCode Config Cleanup
Date: 2026-06-06

Table of Contents

Pass 1: Error Examination 4

1.1 Claude Extended Thinking Blocks Are Silently Dropped 4

1.2 Non-Atomic Python Module Writes (Safe-Mode Violation) 4

1.3 Logrotate ExecStart Shell Variable Not Expanded 5

1.4 JSONC Comment Stripping is Lossy and Dangerous 5

1.5 Port Detection Matches Partial Port Numbers 5

1.6 Kiro Gateway Systemd Service Uses Hardcoded Path 5

1.7 StreamRacer Never Records Success for Winning Provider 6

1.8 Forward Proxy Semaphore Initialized as None 6

1.9 Uninstall Does Not Remove All Artifacts 6

1.10 OpenCode Config Merge Has Fragile Shell Expansion 6

1.11 SIGHUP Signal Handler Safety in Asyncio Context 7

1.12 Orca Router Does Not Handle Non-Streaming Requests 7

1.13 Pass 1 Error Summary 7

Pass 2: Fix Suggestions and Optimization Tweaks 8

2.1 Fix: Add Extended Thinking Block Translation 8

2.2 Fix: Use atomic_write for All Python Module Writes 8

2.3 Fix: Logrotate ExecStart Path Expansion 9

2.4 Fix: Safe JSONC Comment Stripping 9

2.5 Fix: Record Circuit Success for Race Winner 9

2.6 Optimization: Precise Port Detection 10

2.7 Optimization: Dynamic Kiro Gateway Service Path 10

2.8 Optimization: Add Non-Streaming Request Support 10
2.9 Optimization: Add fsync Before mv in atomic_write 10
2.10 Optimization: Add Retry Logic for Pip Installs 10
2.11 Optimization: Async SIGHUP Reload Pattern 11
2.12 Pass 2 Fix and Optimization Summary 11
Pass 3: Enhancements with Constraints and Pushback 11
3.1 Enhancement: Observability Metrics Endpoint 11
3.2 Enhancement: Request Rate Limiting 12
3.3 Enhancement: HTTP/2 for Upstream Provider Connections 12
3.4 Enhancement: Configuration Validation on SIGHUP Reload 13
3.5 Enhancement: Request Correlation IDs 13
3.6 Enhancement: Graceful Degradation When All Circuits Open 14
3.7 Enhancement: Per-Route Streaming Timeouts 14
3.8 Enhancement: Persistent Radix Tree Serialization 14
3.9 Enhancement: Migrate Services to Podman Rootless Containers 15
3.10 Enhancement: Automatic Provider Failover on Auth Expiry 15
3.11 Pass 3 Enhancement Decision Summary 16
Answers to Specific Questions 16
Why Claude's Extended Thinking Blocks Get Dropped 16
OpenCode Config Cleanup: Uninstall Removes owl-orca-virtual Provider 17
Audit Conclusion and Priority Matrix 17

Pass 1: Error Examination

This pass examines the install.sh script (2,692 lines) for bugs, logic errors, edge cases, security issues, and correctness problems. Each finding is classified by severity: CRITICAL (will cause runtime failure or data loss), HIGH (causes incorrect behavior), MEDIUM (subtle bug or degraded behavior), LOW (cosmetic or unlikely edge case).

1.1 Claude Extended Thinking Blocks Are Silently Dropped

Why Claude's extended thinking blocks get dropped: The StreamTranslator.anthropic_sse_to_openai() method in payload_translator.py (line 1048-1116) only handles four Anthropic SSE event types: message_start, content_block_delta, message_delta, and message_stop. When Claude uses extended thinking, it emits content_block_delta events with a "thinking" type delta (delta.thinking) instead of the "text" type delta (delta.text) that the translator expects. Since the code at line 1089 only extracts data.get("delta", {}).get("text", ""), thinking blocks produce an empty string which causes the method to return None at line 1098. This means all of Claude's reasoning content is silently discarded during the Anthropic-to-OpenAI SSE translation, and the client never sees it.

This is a significant semantic loss. Extended thinking is Claude's chain-of-thought reasoning that precedes the actual response. While discarding it makes the output OpenAI-compatible (which has no equivalent field), the user loses visibility into the model's reasoning process. For applications that rely on thinking tokens for debugging or audit trails, this is a data-loss bug. The fix should either pass thinking blocks through as a custom extension field, or optionally strip them with a configuration flag rather than hardcoding the discard behavior.

```
# Current code (line 1089) -- only handles "text" deltas:
text = data.get("delta", {}).get("text", "")
if text:
return "data: " + json.dumps({...}) + "\n\n"
return None # thinking blocks fall through here
```

1.2 Non-Atomic Python Module Writes (Safe-Mode Violation)

The script defines an atomic_write() function (lines 427-443) that writes to a temp file then swaps the inode with mv. This is the core Safe-Mode primitive to prevent file watchers from seeing half-written files. However, ALL Python module writes (Steps 6A-6E) use plain "cat >" which truncates the destination first, then writes content. If an IDE file watcher (OpenCode, Cline, VSCode) sees the IN_MODIFY event between the truncate and the final write, it can crash or read corrupted module code. This directly violates the Safe-Mode guarantee advertised in the script header.

The affected files include: radix_tree.py, circuits.py, forward_proxy.py, payload_translator.py, token_manager.py, orca_router.py, owl_resilient_mcp.py, providers.json, routes.json, and the kiro-gateway .env file. Every single file written with "cat >" instead of atomic_write() is a potential file-watcher crash trigger. The fix is straightforward: replace all "cat > $DEST" calls with atomic_write "$DEST" "$CONTENT" invocations, piping the heredoc content through a variable first.

1.3 Logrotate ExecStart Shell Variable Not Expanded

At line 2624, the logrotate systemd service file contains: ExecStart=/usr/bin/logrotate ${CONFIG_DIR}/logrotate.conf. This heredoc is delimited by single-quoted LRSEOF, which prevents bash from expanding shell variables. Systemd does not perform shell variable substitution on ExecStart lines, so the literal string "${CONFIG_DIR}" will be passed to logrotate, which will fail with "No such file or directory." The CONFIG_DIR variable expands to ~/.owl-agent/config at install time, but this value never reaches the systemd unit file. This is a CRITICAL bug because log rotation will silently fail, and on an 8GB RAM system, unbounded log growth will eventually consume all available disk space.

```
# Line 2624 -- variable NOT expanded (single-quoted heredoc):
ExecStart=/usr/bin/logrotate ${CONFIG_DIR}/logrotate.conf

# Fix -- use double-quoted heredoc or hardcode the path:
ExecStart=/usr/bin/logrotate %h/.owl-agent/config/logrotate.conf
```

1.4 JSONC Comment Stripping is Lossy and Dangerous

The uninstall section (line 169) and the OpenCode config merge (line 2330) both use the regex re.sub(r"//.*?\n|/*.*?*/", "", raw, flags=re.S) to strip JSONC comments before parsing. This regex incorrectly matches // inside JSON string values. For example, a URL like "https://example.com" would be corrupted to "https: and the remainder of the line discarded. Since the OpenCode config contains provider URLs with // in string values, this is a data corruption risk during both installation and uninstallation. Furthermore, after stripping comments, the config is written back as plain minified JSON (using separators=(",",":")), destroying all JSONC formatting, comments, and readability. OpenCode may also rely on JSONC-specific formatting that plain JSON lacks.

The OpenCode config cleanup during uninstall (lines 159-179) has the same problem: it reads JSONC, strips comments with the unsafe regex, and writes back as plain JSON. This answers the user's question about why uninstall should remove the owl-orca-virtual provider: the current implementation does remove it, but in the process it destroys the entire JSONC formatting of the file. A safer approach would be to use a line-by-line approach that only modifies the specific provider key, or to use a proper JSON5/JSONC parser library.

1.5 Port Detection Matches Partial Port Numbers

The check_port_free() function at line 2489 uses: ss -tlnp 2>/dev/null | grep -q ":${port} ". This can produce false positives because it matches any port that starts with the given number. For example, checking port 6000 would match port 60000, 60001, 60002, etc. The trailing space helps slightly but does not solve the problem because ss output format is not guaranteed to have a space after the port number in all cases. A more robust check would use a word-boundary-aware regex like grep -qP ":${port}\b" or parse the ss output properly with awk.

1.6 Kiro Gateway Systemd Service Uses Hardcoded Path

The KIRO_GATEWAY_DIR variable defaults to $HOME/Documents/proxy/kiro-gateway and can be overridden via the OWL_KIRO_DIR environment variable. However, the systemd service file (lines 2252-2269) hardcodes the path as %h/Documents/proxy/kiro-gateway for both WorkingDirectory and ExecStart. If the user overrides OWL_KIRO_DIR, the git clone goes to the custom location but the systemd service still points to the default path, causing the service to fail. The service file must be generated dynamically to reflect the actual KIRO_GATEWAY_DIR value, or the heredoc delimiter must be unquoted to allow shell expansion.

1.7 StreamRacer Never Records Success for Winning Provider

In _handle_single_stream() (line 1892), circuit.record_success() is called after a successful stream completion. However, in _handle_race() (lines 1804-1839), the winning stream's circuit breaker never has record_success() called. This means that even though a provider wins a race and serves the response successfully, its circuit breaker never transitions from HALF_OPEN back to CLOSED. Over time, this causes circuit breakers to remain stuck in OPEN or HALF_OPEN state even for healthy providers, effectively reducing the available provider pool and defeating the circuit breaker recovery mechanism.

The fix is to track which provider won the race and call circuit.record_success() for that provider after the race completes. This requires minor refactoring of the StreamRacer.race() method to return the winning stream's index alongside the translated chunks.

1.8 Forward Proxy Semaphore Initialized as None

The _conn_semaphore global variable at line 734 is initialized to None and only set to an asyncio.Semaphore in main() at line 934. If handle_connect() or handle_http() is called before main() executes (which should not happen in normal flow, but could occur during testing or if the module is imported), the "async with _conn_semaphore" statement will raise TypeError: "NoneType object does not support asynchronous context manager." This is a defensive programming issue that could be fixed by initializing the semaphore at module level or adding a None check in the handlers.

1.9 Uninstall Does Not Remove All Artifacts

The uninstall routine (lines 114-184) removes the three main services and CLI wrappers, but misses several artifacts that were created during installation: (1) The owl-logrotate.service and owl-logrotate.timer systemd units are not stopped, disabled, or removed; (2) The logrotate.conf file in $CONFIG_DIR is orphaned (though it would be caught by rm -rf $INSTALL_DIR); (3) The MCP server configuration in ~/.config/opencode/mcp.json is not cleaned up; (4) The owl_resilient_mcp.py script reference remains in MCP config after uninstall; (5) The Kiro Gateway directory ($HOME/Documents/proxy/kiro-gateway) is not removed. While the INSTALL_DIR removal catches most files, the external artifacts in ~/.config/opencode/ and the systemd user directory are partially orphaned.

1.10 OpenCode Config Merge Has Fragile Shell Expansion

At line 2321, the Python heredoc for config merging uses an unquoted delimiter (PYEOF without single quotes), which means bash expands shell variables inside it. The line cfg_path = "$OPENCODE_CONFIG" relies on this expansion to get the actual path. However, the line new_block = json.loads('''$NEW_CONFIG_BLOCK''') also relies on shell expansion of $NEW_CONFIG_BLOCK. If the JSON content contains shell special characters like backticks, dollar signs, or exclamation marks, the expansion will corrupt the data or cause a bash syntax error. While the current JSON content is safe, this is a fragile pattern that will break if any provider URL or model name contains shell metacharacters in the future.

1.11 SIGHUP Signal Handler Safety in Asyncio Context

At line 1683, signal.signal(signal.SIGHUP, self._handle_sighup) registers a synchronous signal handler. The handler rebuilds the radix tree and reloads configuration synchronously. In CPython, signal handlers are called from the main thread between bytecode instructions, which means this is safe for asyncio as long as the handler does not block or perform I/O. The current implementation only reads JSON files and rebuilds in-memory data structures, which is fast enough to be safe. However, if the config files are large or on a slow filesystem, the signal handler could block the event loop. A safer pattern would be to set a threading.Event in the signal handler and process the reload in an asyncio task.

1.12 Orca Router Does Not Handle Non-Streaming Requests

The handle_chat() handler in orca_router.py (line 1944) always returns a StreamResponse with Content-Type: text/event-stream, regardless of whether the client requested streaming or not. If a client sends a request with "stream": false, it receives SSE-formatted data anyway. While most LLM clients use streaming, non-streaming requests are valid and should return a standard JSON response. This is a correctness issue that could cause integration problems with clients that expect a standard OpenAI JSON response for non-streaming completions.

1.13 Pass 1 Error Summary

Severity	ID	Description
CRITICAL	1.3	Logrotate ExecStart variable not expanded; rotation silently fails
CRITICAL	1.2	Non-atomic Python module writes violate Safe-Mode guarantee
HIGH	1.1	Claude extended thinking blocks silently dropped during SSE translation
HIGH	1.4	JSONC comment stripping regex corrupts URLs and string values
HIGH	1.7	StreamRacer never records circuit success for winning provider
HIGH	1.6	Kiro Gateway systemd path hardcoded; breaks with OWL_KIRO_DIR override
MEDIUM	1.5	Port detection matches partial port numbers (6000 matches 60000)
MEDIUM	1.9	Uninstall misses logrotate services, MCP config, Kiro directory

Severity	ID	Description
MEDIUM	1.10	OpenCode config merge uses fragile shell variable expansion
MEDIUM	1.12	Orca Router always returns SSE even for non-streaming requests
LOW	1.8	Forward proxy semaphore initialized as None; crash if called before main()
LOW	1.11	SIGHUP signal handler is synchronous; could block event loop on slow I/O

Pass 2: Fix Suggestions and Optimization Tweaks

This pass provides concrete fixes for the errors identified in Pass 1, along with performance and reliability optimizations. Each fix includes the rationale, the specific code change, and any trade-offs or constraints that apply.

2.1 Fix: Add Extended Thinking Block Translation

To properly handle Claude's extended thinking blocks, the StreamTranslator.anthropic_sse_to_openai() method needs an additional case for thinking content_block_delta events. The recommended approach is to emit thinking content as a custom OpenAI extension field, which preserves the data without breaking the OpenAI SSE format. Clients that understand the extension can display thinking; those that do not will simply ignore the extra field. This is preferable to silently discarding the data.

```
# Add after the content_block_delta "text" handler (line 1098):
# 2b. content_block_delta (thinking) - pass through as custom extension
thinking = data.get("delta", {}).get("thinking", "")
if thinking:
return "data: " + json.dumps({
"id": "owl-orca-msg",
"object": "chat.completion.chunk",
"created": int(time.time()),
"model": "orca-translated",
"choices": [{"index": 0, "delta": {
"content": "",
"thinking": thinking # Custom extension
}, "finish_reason": None}],
}) + "\n\n"
```

Trade-off: Adding a non-standard "thinking" field to the OpenAI chunk format means strict OpenAI API validators may reject the response. However, the OpenAI Python SDK and most clients simply ignore unknown fields. An alternative is to add a configuration flag TRANSLATE_THINKING=true/false that defaults to stripping thinking blocks but can be enabled when the client supports it. This gives the user control over the trade-off.

2.2 Fix: Use atomic_write for All Python Module Writes

Replace all "cat > $DEST" patterns with atomic_write invocations. Since atomic_write takes the content as a string argument, the heredoc content must first be captured into a shell variable. This requires changing from "cat >" heredocs to variable-then-atomic_write patterns. The key change is: instead of cat

> "$DEST" << 'PYEOF' ... PYEOF, use read -r -d '' CONTENT << 'PYEOF' ... PYEOF; atomic_write "$DEST" "$CONTENT". However, bash variables cannot contain NUL bytes, which is not a concern for Python source code. For large files, this approach consumes more RAM temporarily (the content is held in a variable), but since the largest module (orca_router.py) is approximately 15KB, this is negligible even on an 8GB RAM system.

Trade-off: The variable-based approach is slightly more complex and uses more memory than direct file writes, but the memory overhead is trivial (under 50KB total for all modules). The correctness benefit of preventing file-watcher crashes far outweighs the minimal complexity cost. An alternative is to use a temp-file-then-mv pattern inline (write to $DEST.owl_tmp then mv), which is exactly what atomic_write already does.

2.3 Fix: Logrotate ExecStart Path Expansion

Change the logrotate service heredoc delimiter from single-quoted LRSEOF to an unquoted delimiter, allowing shell variable expansion. Alternatively, hardcode the expanded path using systemd's %h specifier (which expands to the user's home directory). The systemd specifier approach is preferred because it is independent of the shell's variable state at install time.

```
# Current (broken):
ExecStart=/usr/bin/logrotate ${CONFIG_DIR}/logrotate.conf

# Fix (using systemd specifier):
ExecStart=/usr/bin/logrotate %h/.owl-agent/config/logrotate.conf
```

2.4 Fix: Safe JSONC Comment Stripping

The regex-based JSONC comment stripping is fundamentally unsafe because it cannot distinguish between // in code comments and // inside string values. The recommended fix is to use a simple state-machine parser that tracks whether the parser is inside a string literal. This is a well-known problem with a known solution. Alternatively, since Python 3.9+, the json5 library can parse JSONC natively, but adding a dependency is a constraint. The state-machine approach adds approximately 20 lines of Python but handles all edge cases correctly.

Additionally, the config merge should preserve JSONC formatting. Since the installer is the only tool modifying opencode.jsonc, a line-by-line approach that only adds/removes the specific provider block would be far safer than parsing and re-serializing the entire file. This preserves comments, whitespace, and any custom configuration the user has added to the file. The trade-off is slightly more complex code, but the benefit is that users do not lose their hand-crafted JSONC configuration when the installer runs.

2.5 Fix: Record Circuit Success for Race Winner

Modify StreamRacer.race() to return the winning stream index alongside each chunk. This can be done by wrapping chunks in a (stream_id, chunk) tuple. The caller (_handle_race) then tracks the first stream_id it sees (the winner) and calls circuit.record_success() for that provider after the race completes. Alternatively, the race() method can accept a callback that is called once with the winner's stream_id. This is a minimal change that restores correct circuit breaker behavior for raced providers.

2.6 Optimization: Precise Port Detection

Replace the grep-based port check with a more precise pattern that uses word boundaries. The fix is to change the grep pattern from ":${port} " to a pattern that matches the port only when followed by a non-digit character or end-of-line. Using ss output parsing with awk is more reliable than grep for this purpose.

```
# Current (imprecise):
ss -tlnp 2>/dev/null | grep -q ":${port} "

# Fix (precise with word boundary):
ss -tlnp 2>/dev/null | grep -qP ":${port}\b"
```

2.7 Optimization: Dynamic Kiro Gateway Service Path

Use an unquoted heredoc delimiter for the kiro-gateway.service file so that shell variables are expanded, or dynamically generate the path using the KIRO_GATEWAY_DIR variable. The safest approach is to use a hybrid: use systemd %h for the home directory but embed the actual relative path from KIRO_GATEWAY_DIR. Since the service file is generated after git clone, the directory is guaranteed to exist at that point.

2.8 Optimization: Add Non-Streaming Request Support

Check the "stream" field in the incoming request payload. If false, collect the full response from the async generator and return it as a standard JSON response instead of SSE. This adds a small memory overhead for non-streaming responses (the full response is buffered), but it is the correct behavior for OpenAI API compatibility. For streaming responses, the current zero-copy SSE behavior is preserved.

2.9 Optimization: Add fsync Before mv in atomic_write

The current atomic_write() writes to a temp file and then moves it. However, without an fsync on the temp file before the mv, a power failure between write and mv could result in an empty or partially-written temp file being moved into place. Adding sync; sync before the mv, or using Python's os.fsync() if the write is done from Python, ensures durability. In the bash context, the simplest fix is to add "sync" before the "mv" command, though this has a performance cost. Since the script only writes a few files during installation, the cost is negligible.

2.10 Optimization: Add Retry Logic for Pip Installs

The pip install at line 398 has no retry logic. Network issues during installation can cause the entire script to fail. Adding a simple retry loop with 3 attempts and exponential backoff would make the installer more resilient to transient network failures. This is especially important because the script runs with set -e, so any pip failure immediately terminates the installation. The retry logic should only apply to the pip install commands, not to the venv creation or validation steps.

2.11 Optimization: Async SIGHUP Reload Pattern

Replace the synchronous signal.signal(SIGHUP, handler) pattern with an asyncio-safe alternative. The recommended approach is to set a threading.Event in the signal handler and have an asyncio task that waits on the event and performs the reload in the event loop. This ensures the reload does not block the event loop, which is important for maintaining the zero-downtime guarantee during configuration changes. The code change is minimal: replace signal.signal(SIGHUP, handler) with loop.add_signal_handler(SIGHUP, self._schedule_reload) where _schedule_reload creates an asyncio task for the actual reload work.

2.12 Pass 2 Fix and Optimization Summary

Fix ID	Maps To	Description	Priority
2.1	1.1	Add extended thinking block translation with custom extension field	HIGH
2.2	1.2	Use atomic_write for all Python module writes	HIGH
2.3	1.3	Use systemd %h specifier for logrotate path	CRITICAL
2.4	1.4	Replace regex JSONC parser with state-machine or line-by-line approach	HIGH
2.5	1.7	Return winner stream_id from StreamRacer; record_success() for winner	HIGH
2.6	1.5	Use grep -P with word boundary for port detection	MEDIUM
2.7	1.6	Use unquoted heredoc for kiro-gateway.service	HIGH
2.8	1.12	Check "stream" field; return JSON for non-streaming requests	MEDIUM
2.9	New	Add fsync before mv in atomic_write for durability	LOW
2.10	New	Add retry logic (3 attempts) for pip install commands	MEDIUM
2.11	1.11	Replace signal.signal with loop.add_signal_handler for async safety	LOW

Pass 3: Enhancements with Constraints and Pushback

This pass proposes enhancements to the installer and runtime system. Each proposal is challenged against the 8GB RAM constraint, the zero-downtime requirement, and the principle of keeping the installer self-contained. Pushback is provided for each enhancement to validate whether it truly adds value or introduces unnecessary complexity.

3.1 Enhancement: Observability Metrics Endpoint

Proposal: Add a /metrics endpoint to the Orca Router that exposes request counts, latency histograms, circuit breaker states, and provider success/failure rates in Prometheus format. This would enable operators to monitor the health and performance of the routing layer with standard observability tools like Grafana.

Pushback: Prometheus exposition format requires the prometheus_client library, which adds a dependency and consumes memory for metric aggregation. On an 8GB RAM system with MemoryMax=384M for the router, every MB counts. A full Prometheus client with histogram buckets could consume 10-20MB of RSS, which is significant. Additionally, the current /health endpoint already provides circuit breaker status and route information in JSON format.

Compromise: Instead of Prometheus format, extend the existing /health endpoint with optional query parameters like ?metrics=true that returns lightweight JSON counters (request_count, error_count, avg_latency_ms, circuit_states). This adds observability without a new dependency or significant memory overhead. The data can be scraped by a simple curl script or fed into any monitoring system that accepts JSON. This approach keeps the installer self-contained while providing meaningful operational visibility.

3.2 Enhancement: Request Rate Limiting

Proposal: Add per-client and global rate limiting to the Orca Router to protect against accidental or malicious request floods. This is particularly important because the router provides free-tier LLM access through Copilot and Antigravity, and a single runaway client could exhaust the provider's rate limits for all users.

Pushback: Rate limiting adds complexity and state management. Per-client limiting requires tracking request counts per IP or API key, which consumes memory proportional to the number of unique clients. On a single-user workstation (the primary deployment target), there is typically only one client (OpenCode), making per-client limiting unnecessary. Global limiting could be useful, but the upstream providers already enforce their own rate limits, so the router's rate limiting would be redundant.

Compromise: Implement a simple global concurrency limit using asyncio.Semaphore (similar to the forward proxy's existing MAX_CONNECTIONS pattern). This prevents the router from opening too many simultaneous upstream connections without adding per-client tracking overhead. The limit can be configured via an environment variable (OWL_MAX_CONCURRENT_REQUESTS=10 by default). This is a 5-line change that provides meaningful protection without the complexity of token-bucket rate limiting. Per-client limiting should only be added if multi-user deployment becomes a requirement.

3.3 Enhancement: HTTP/2 for Upstream Provider Connections

Proposal: Enable HTTP/2 for upstream connections to providers that support it (Copilot, Anthropic). HTTP/2 multiplexing allows multiple concurrent streams over a single TCP connection, reducing latency and connection overhead. The httpx library already supports HTTP/2 via the httpx[http2] extra, which is already installed (line 398).

Pushback: HTTP/2 connection pooling with httpx creates long-lived connections that consume memory per connection. The current per-request AsyncClient pattern (opening and closing a client for each

request) is simple and safe, but it cannot benefit from HTTP/2 multiplexing because each request creates a new connection. To benefit from HTTP/2, the router would need a persistent httpx.AsyncClient that is shared across requests, which requires lifecycle management (reconnection on failure, connection pooling limits, cleanup on shutdown). This adds significant complexity.

Compromise: Create a shared httpx.AsyncClient as a singleton on the OrcaRouter class with HTTP/2 enabled and a connection pool limit of 5. This client is created once during __init__ and reused for all requests. The memory overhead is minimal (one connection pool instead of per-request clients), and the latency improvement for Stream Racing is significant because the first-byte time includes TCP+TLS handshake for each new connection. With a persistent client, the handshake is amortized across requests. The client should be properly closed during shutdown.

3.4 Enhancement: Configuration Validation on SIGHUP Reload

Proposal: Validate the new configuration before applying it during SIGHUP reload. Currently, if a malformed routes.json or providers.json is written to disk and then SIGHUP is sent, the router will load the broken config and rebuild the radix tree with empty or invalid routes, effectively breaking all routing until the config is fixed and another SIGHUP is sent.

Pushback: Full config validation (checking that all provider references in routes exist in providers, verifying URL formats, validating weight sums) adds code complexity and could be over-engineering for a single-user tool. The current fallback behavior (loading default config if the file is invalid) is already a form of validation.

Compromise: Add minimal validation that checks: (1) the file parses as valid JSON, (2) the "routes" key exists and is a list, (3) each route has "pattern" and "targets" keys, (4) all provider names in targets exist in the providers dict. If validation fails, log a warning and keep the current configuration instead of applying the broken one. This is approximately 15 lines of Python that prevent the most common config errors without over-engineering.

3.5 Enhancement: Request Correlation IDs

Proposal: Add a unique correlation ID (UUID) to each request that flows through the router. This ID would be included in all log messages for that request, making it possible to trace a single request from receipt through provider selection, upstream fetch, translation, and response. Without correlation IDs, debugging a failed request requires correlating timestamps across multiple log lines, which is error-prone.

Pushback: Adding a correlation ID requires threading it through all function calls and log statements, which is a significant refactoring effort across orca_router.py. The memory overhead is negligible (one UUID string per request), but the code change touches nearly every function. For a single-user workstation where only one request is active at a time, the benefit is marginal compared to a server handling hundreds of concurrent requests.

Compromise: Use Python's logging.LoggerAdapter to automatically inject a request-scoped correlation ID into all log messages for that request. This requires minimal code changes: create the adapter at the start of handle_chat() and pass it to sub-functions. The adapter adds the correlation ID as a prefix to

every log message. This is a clean pattern that does not require threading the ID through every function signature. Implementation is approximately 10 lines of code change and provides significant debugging value.

3.6 Enhancement: Graceful Degradation When All Circuits Open

Proposal: When all provider circuits are open, instead of returning a 500 error, return a meaningful error response that includes: (1) which circuits are open, (2) how long until the next probe attempt, and (3) a suggested action for the user. This improves the user experience during provider outages.

Pushback: This is a nice-to-have feature, but the current behavior (raising an exception that becomes an orca_error JSON response) already provides basic error information. Adding circuit state to error responses could leak operational details that are not relevant to the end user. The OpenCode client that receives the error cannot do anything with circuit breaker state information -- it will simply retry or show an error message.

Compromise: Add circuit state to the error response only when the request includes a debug header (X-Owl-Debug: true) or when the router is running in a debug log level. This keeps the default error response clean while providing detailed information when specifically requested. The implementation is straightforward: check the debug flag in the exception handler and conditionally include the circuit state dict in the error JSON. This is a 10-line change that provides diagnostic value without polluting normal error responses.

3.7 Enhancement: Per-Route Streaming Timeouts

Proposal: Add configurable timeouts for upstream streaming responses. Currently, the httpx client uses a fixed 120-second timeout (line 1793). If a provider hangs after sending the first chunk, the connection remains open until the 120-second timeout expires, consuming a connection slot and potentially exhausting the circuit breaker's failure threshold with a single hung request.

Pushback: Adding per-route timeouts requires extending the routes.json schema and the radix tree handler dict. For a simple two-route configuration (v1/chat/completions and v1/models), this adds complexity with minimal benefit. The current 120-second timeout is reasonable for LLM requests, and the circuit breaker already handles provider failures. A hung connection after first byte is an edge case that the 120-second timeout adequately handles.

Compromise: Keep the fixed timeout but reduce it from 120s to 60s for the connect phase (establishing the connection) while keeping 120s for the read phase (receiving data). httpx supports separate connect and read timeouts via httpx.Timeout(connect=60.0, read=120.0). This is a one-line change that addresses the most common timeout issue (slow DNS/TCP handshake) without adding per-route configuration complexity.

3.8 Enhancement: Persistent Radix Tree Serialization

Proposal: Serialize the radix tree to a binary format (pickle or msgpack) after loading routes, and reload from the serialized format on subsequent starts. This would reduce startup time from JSON parse + tree

build to a single deserialization call.

Pushback (STRONG): This enhancement is not worth implementing. The route count is currently 2 (v1/chat/completions and v1/models). Building a radix tree with 2 entries takes microseconds -- there is no performance problem to solve. Serialization adds complexity (cache invalidation, format versioning, pickle security concerns), and the benefit is measured in nanoseconds. Even if the route count grows to 100, the radix tree build time would still be under 1ms. This is a classic case of premature optimization. The JSON-based route loading is human-readable, debuggable, and fast enough. **Recommendation: Reject this enhancement.**

3.9 Enhancement: Migrate Services to Podman Rootless Containers

Proposal: Run the Orca Router, Forward Proxy, and Kiro Gateway as Podman rootless containers instead of systemd user services. This provides stronger isolation, reproducible deployments, and easier dependency management. The installer already installs Podman (line 285-296) and the v6.2 specification mentioned Podman rootless as the deployment target.

Pushback (STRONG): Podman rootless containers add significant complexity without meaningful benefit for a single-user workstation deployment. The current systemd user services are simpler, faster to start, easier to debug (journalctl), and already provide process isolation via systemd's security features (MemoryMax, PrivateTmp, etc.). Podman containers require building and maintaining container images, managing volume mounts for config/logs, handling networking (host network vs port mapping), and debugging inside containers. The memory overhead of Podman itself (conmon, rootlesskit, slirp4netns) can be 50-100MB, which is significant on an 8GB RAM system. The installer's promise of self-contained simplicity would be undermined by container management. **Recommendation: Reject for primary services; keep Podman as optional for sidecar containers only.**

3.10 Enhancement: Automatic Provider Failover on Auth Expiry

Proposal: When a provider's authentication token expires or becomes invalid (detected via 401/403 responses), automatically trigger re-authentication using the TokenManager and retry the request with the fresh token. This would prevent service interruptions when Copilot or Antigravity tokens expire during a session.

Pushback: Automatic re-authentication for the Copilot device flow requires user interaction (opening a browser and entering a code), which cannot be done automatically. The Antigravity OAuth flow also requires browser interaction. Only API-key-based auth (Antigravity with --api-key) could be automatically refreshed, and API keys typically have long expiry (365 days in the current implementation). The common case where tokens expire is Copilot, which cannot be auto-refreshed. Implementing this for the uncommon case adds complexity without addressing the primary failure mode.

Compromise: Instead of automatic re-authentication, add a 401/403 detection hook in _fetch_stream() that logs a clear message telling the user to run "owl-token auth --provider X" and temporarily opens the circuit breaker for that provider to route traffic to alternatives. This provides a graceful degradation path without the complexity of automated interactive auth. The implementation is approximately 5 lines of

code in the exception handler of _fetch_stream().

3.11 Pass 3 Enhancement Decision Summary

ID	Enhancement	Decision	Rationale
3.1	Observability Metrics	Compromise	Extend /health with JSON metrics; skip Prometheus library
3.2	Request Rate Limiting	Compromise	Global concurrency limit via Semaphore; skip per-client limiting
3.3	HTTP/2 Upstream	Compromise	Shared httpx.AsyncClient with HTTP/2; skip per-request clients
3.4	Config Validation on SIGHUP	Compromise	Minimal JSON parse + key existence check; keep old config on failure
3.5	Request Correlation IDs	Compromise	LoggerAdapter for request-scoped ID; skip threading through functions
3.6	Graceful Degradation	Compromise	Debug-header-gated circuit state in error response
3.7	Streaming Timeouts	Compromise	Separate connect=60s / read=120s timeouts via httpx.Timeout
3.8	Persistent Radix Tree	REJECTED	Premature optimization; 2 routes build in microseconds
3.9	Podman Containerization	REJECTED	Adds 50-100MB overhead; undermines simplicity promise
3.10	Auto Provider Failover	Compromise	Log re-auth hint + open circuit; skip automated interactive auth

Answers to Specific Questions

Why Claude's Extended Thinking Blocks Get Dropped

Claude's extended thinking blocks are dropped because the StreamTranslator.anthropic_sse_to_openai() method only handles content_block_delta events where the delta contains a "text" key. When Claude uses extended thinking, it emits content_block_delta events with a "thinking" key instead. The current code at line 1089 extracts data.get("delta", {}).get("text", ""), which returns an empty string for thinking blocks, causing the method to return None at line 1098, effectively discarding the thinking content. This is not a bug in the sense of incorrect logic -- the code correctly translates text deltas -- but it is a missing feature that causes data loss for an important Anthropic capability. The fix (described in Section 2.1) is to add a handler for thinking deltas that passes them through as a custom extension in the OpenAI chunk format.

The deeper architectural question is whether thinking blocks SHOULD be translated at all. The OpenAI API has no equivalent to extended thinking, so any translation is inherently non-standard. Three options exist: (1) Silently discard (current behavior -- loses data), (2) Pass through as a custom extension (recommended -- preserves data with compatibility risk), or (3) Buffer thinking content and prepend it to the first text chunk (loses the streaming/stream-of-consciousness nature of thinking). Option 2 is recommended because it preserves the most information while maintaining backward compatibility -- clients that do not understand the "thinking" field will simply ignore it.

OpenCode Config Cleanup: Uninstall Removes owl-orca-virtual Provider

The uninstall routine (lines 159-179) correctly removes the owl-orca-virtual provider from opencode.jsonc, but it does so in a way that destroys the file's formatting. The Python cleanup script reads the JSONC file, strips all comments using an unsafe regex, parses the result as JSON, deletes the "owl-orca-virtual" key from the providers dict, and writes back as plain minified JSON. This means: (1) All JSONC comments (including user-added ones) are destroyed, (2) All formatting and whitespace is lost, (3) The file changes from JSONC to plain JSON, which may confuse tools that expect JSONC syntax. The cleanup is functionally correct (the provider IS removed) but destructive to the file's structure.

The recommended improvement is to use a line-by-line approach for the uninstall cleanup that only removes the specific owl-orca-virtual provider block while preserving all other content, comments, and formatting. This can be done by finding the start of the "owl-orca-virtual" key and removing lines until the matching closing brace. While this is more complex than the JSON parse-modify-write approach, it preserves the user's hand-crafted configuration, which is critical for a config file that the user may have extensively customized. An even simpler approach: use Python's json module but write back with indent=2 and preserve the .jsonc extension, accepting that comments will be lost but at least the formatting will be readable. This is a reasonable compromise between correctness and implementation complexity.

Audit Conclusion and Priority Matrix

The OWL-ORCA Master Installer v7.1.0 is a well-structured and feature-rich script that successfully merges multiple specification versions into a single coherent installer. The overall architecture is sound: atomic writes, circuit breakers, radix tree routing, and stream racing are all correctly implemented at the conceptual level. However, the three-pass audit has identified 12 errors (2 critical, 4 high, 4 medium, 2 low), 11 fix/optimization suggestions, and evaluated 10 enhancement proposals (2 rejected, 8 compromised). The following priority matrix summarizes the recommended implementation order.

Priority	Severity	Action Item
P0	CRITICAL	Fix logrotate ExecStart variable expansion (2.3)
P0	CRITICAL	Replace all cat > with atomic_write for Safe-Mode compliance (2.2)

Priority	Severity	Action Item
P1	HIGH	Add extended thinking block translation (2.1)
P1	HIGH	Fix JSONC comment stripping with safe parser (2.4)
P1	HIGH	Fix StreamRacer missing record_success (2.5)
P1	HIGH	Fix Kiro Gateway hardcoded service path (2.7)
P2	MEDIUM	Fix port detection partial matching (2.6)
P2	MEDIUM	Add non-streaming request support (2.8)
P2	MEDIUM	Add pip install retry logic (2.10)
P2	MEDIUM	Fix uninstall to remove all artifacts (1.9)
P3	LOW	Add fsync to atomic_write (2.9)
P3	LOW	Replace signal.signal with async-safe pattern (2.11)
P4	LOW	Initialize forward proxy semaphore eagerly (1.8)